DEPI | Digital Egypt Pioneers Initiative

Food Rescue

Project Documentation

Project Planning • Stakeholder Analysis • Database Design • UI/UX Design

Item	Details
Track	Front-End Development
Project	Food Rescue – a web platform connecting surplus food donors with charities and families in need
Team Leader	Mohamed AbdelHady Fathy Mohamed
Team members	Omer Mohamed AbdelQader – Front-End Developer (Donor side) Youssef Mohamed Ibrahim – Front-End Developer (Receiver side) Yousef Hesham Anwar – Back-End Developer Ahmed Mostafa Ali Mostafa – UI/UX, QA & Documentation
Instructor	Mohamed Abdel Salam
Repository	https://github.com/nhahub/NHA-5-061
Version / Date	v1.0 – October 2026

Roles are assigned by the team and can be adjusted with the instructor.

1. Project Planning & Management

1.1 Project overview

Food Rescue is a web platform that connects restaurants and stores holding surplus edible food (donors) with charities and families in need (receivers). Donors post available food with its quantity, pickup window and location. Receivers browse nearby donations on a map, reserve them and confirm pickup. After each handover both sides rate each other, and dashboards show meals saved and estimated environmental impact.

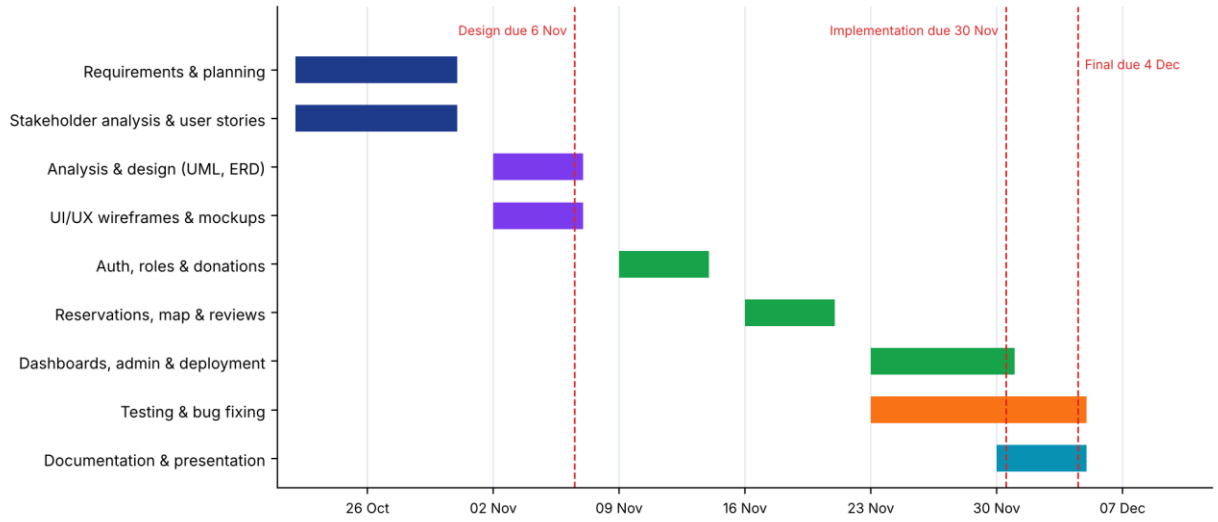
Problem context: Egypt wastes roughly 9 million tons of food per year according to the UNEP Food Waste Index, while many households face food insecurity. Food Rescue is a technical bridge between the two.

1.2 Objectives and scope

- Reduce edible food waste by making surplus food visible to the people who can use it.
- Give donors a fast way to publish a donation, and receivers a reliable way to find and reserve food nearby.
- Build trust through ratings, reviews and a moderated environment (Admin role).
- Measure impact with dashboards: meals saved, estimated environmental impact and top donors.
- Out of scope for this release: online payments, native mobile apps, delivery logistics and real-time chat (listed as future work).

1.3 Project plan (Gantt chart)

The plan follows the DEPI documentation calendar: system analysis and design due 6 November, implementation due 30 November, and final presentation, testing and reports due 4 December 2026.



1.4 Milestones and deliverables

Date	Milestone	Deliverables
30 Oct 2026	Requirements & planning complete	Approved proposal, requirements document, project plan, risk plan, KPIs, GitHub repository
6 Nov 2026	Analysis & design complete	Use case, DFD, sequence, activity, state and class diagrams; ERD and schema; wireframes and mockups; API specification
13 Nov 2026	Authentication & donations	Registration and login with roles; donation CRUD; donor dashboard; landing page
20 Nov 2026	Reservations, map & reviews	Reservation flow; map with filters; ratings and reviews; in-app notifications
30 Nov 2026	Implementation complete	Impact dashboard; admin panel; deployed application; README and API documentation
4 Dec 2026	Final delivery	Test report and bug log; user manual; technical documentation; presentation (PDF) and infographic; demo video (optional)

1.5 Task assignment and roles

#	Role	Main responsibilities	Member
1	Team Leader & Front-End Lead	Coordination and communication with the instructor, GitHub repository and branching, React architecture, authentication UI, admin panel, final presentation	Mohamed AbdelHady Fathy Mohamed
2	Front-End Developer (Donor side)	Donor dashboard, add-donation form, impact dashboard and charts	Yousef Hesham Anwar
3	Front-End Developer (Receiver side)	Browse donations, filters and map, reservations, receiver dashboard, profile, notifications page	Youssef Mohamed Ibrahim
4	Back-End Developer	Node/Express REST API, MongoDB schema, JWT and role-based access, stats and notifications, deployment of the API	Omer Mohamed AbdelQader
5	UI/UX, QA & Documentation	Wireframes and mockups, UI guidelines, test plan and bug reports, user manual, technical documentation, infographic	Ahmed Mostafa Ali Mostafa

The Team Leader speaks on behalf of the team and uploads project files to the official GitHub repository. In the final discussion each member explains the work they personally delivered.

1.6 Risk assessment and mitigation

ID	Risk	Likelihood	Impact	Mitigation
R1	Short timeline: implementation window is about three weeks	High	High	Prioritize must-have features; build in vertical slices; weekly progress check; move nice-to-have items to future work
R2	Team members unfamiliar with back-end technologies	Medium	High	Pair a front-end member with the back-end developer; use a managed database (MongoDB Atlas) and Postman collections; fall back to a hosted backend service if needed
R3	Merge conflicts and broken main branch	Medium	Medium	Feature branches, pull-request reviews, protected main/develop branches
R4	Map or third-party service limits or downtime	Low	Medium	Use Leaflet with OpenStreetMap tiles; cache coordinates; show list view as fallback
R5	Unsafe or expired food listed	Medium	High	Mandatory expiry time and pickup window; auto-expire listings; report button and admin moderation; disclaimer on safe handling
R6	Fake accounts or misuse	Medium	Medium	Role-based access, input validation, rate limiting, admin ability to suspend accounts
R7	Scope creep from extra features	Medium	Medium	Change requests approved by the Team Leader; extra features only after core flow works
R8	A member becomes unavailable	Low	High	Shared code ownership, documented tasks on Trello, clear handover notes
R9	Deployment issues close to the deadline	Medium	High	Deploy an early version in Week 3 and redeploy after every milestone

1.7 Key performance indicators (KPIs)

Targets below are proposed by the team and will be reviewed with the instructor.

KPI	Proposed target	How it is measured
Feature completion	100% of must-have requirements implemented by 30 Nov	Requirements checklist in the repository
API response time	95% of requests under 500 ms	Postman / browser network tests on the deployed API
Front-end quality	Lighthouse: performance 80+, accessibility 90+	Lighthouse report on main pages
Test pass rate	90%+ of test cases pass; no open critical bugs at final	Test report and bug log

KPI	Proposed target	How it is measured
Core flow usability	A donor posts a donation in under 2 minutes in user testing	User acceptance testing with sample users
User acceptance	At least 5 test users complete the donor and receiver flows	UAT sheet with feedback
Responsiveness	All screens usable from 360 px to 1920 px width	Manual checks and browser dev tools
Availability	Deployed demo reachable during the evaluation period	Deployment link checked before the presentation

2. Requirements Gathering & Stakeholder Analysis

2.1 Stakeholder analysis

Stakeholder	Type	Needs and expectations	Influence
Donors (restaurants, hotels, supermarkets, individuals)	Primary user	Publish surplus food in under two minutes; know it will be collected on time; reduce waste and disposal cost; receive recognition for social impact	High
Receivers (charities, associations, families in need)	Primary user	Find nearby food quickly; see quantity, expiry and pickup time clearly; reserve with confidence; rate the experience	High
Platform administrator	Operator	Manage users and listings; moderate reports; monitor platform-wide statistics	High
Project team	Builder	Clear requirements, realistic scope, shared plan	High
Instructor and DEPI evaluators	Reviewer	Complete documentation, working system, individual contributions explained	High
Food banks and NGOs	Future partner	Integration with existing distribution networks; reliable reports	Medium
Volunteers and drivers	Future user	Request pickup or delivery tasks (future work)	Low

2.2 User stories

ID	As a…	I want to…	Priority
US1	Visitor	Create an account and choose to be a donor or a receiver, so I see the right screens	Must
US2	Donor	Post a donation with food type, quantity, expiry time, pickup window, photo and location	Must
US3	Donor	Edit or cancel my donation while it is still available	Must
US4	Donor	See my active donations and pending pickups on a dashboard	Must
US5	Receiver	Browse available donations with filters for food type, distance and expiry	Must
US6	Receiver	View donations on a map to see what is closest to me	Must
US7	Receiver	Reserve a donation and see its status (pending, confirmed, completed)	Must
US8	Donor	Confirm a reservation and mark it completed after pickup	Must

ID	As a…	I want to…	Priority
US9	Donor / Receiver	Rate and review the other party after a completed handover	Should
US10	User	Receive notifications about new nearby donations and reservation updates	Should
US11	User	Read an impact dashboard with meals saved and top donors	Should
US12	Admin	Manage users, listings and reported content	Should
US13	User	Edit my profile and see my history	Could

2.3 Use cases

ID	Use case	Actor	Summary
UC1	Register / Log in	Visitor	Create an account with a role or sign in with email and password; receives a JWT session
UC2	Post donation	Donor	Fill the donation form and publish it; status becomes available
UC3	Manage donation	Donor	Edit, cancel or delete an available donation
UC4	Browse and filter donations	Receiver	Search the list or map by food type, distance and expiry
UC5	Reserve donation	Receiver	Select a donation and reserve it; the donation becomes reserved and the donor is notified
UC6	Confirm and complete pickup	Donor	Confirm the reservation, then mark it completed after handover
UC7	Cancel reservation	Receiver / Donor	Cancel a pending or confirmed reservation; the donation becomes available again
UC8	Rate and review	Donor / Receiver	Submit a 1 to 5 rating and comment after completion
UC9	View notifications	All users	Read, and mark as read, system notifications
UC10	View impact dashboard	All users	See meals saved, impact estimates and leaderboard
UC11	Moderate platform	Admin	Review users, listings and reports; suspend or remove content

2.4 Functional requirements

ID	Requirement
FR1	The system shall allow registration and login with email and password, with a role of donor or receiver.

ID	Requirement
FR2	The system shall restrict each screen and API endpoint according to the user role (donor, receiver, admin).
FR3	Donors shall be able to create, view, edit and delete their own donations.
FR4	Each donation shall include food type, description, quantity and unit, expiry time, pickup time window, image and location.
FR5	The system shall automatically mark donations as expired when the expiry time passes.
FR6	Receivers shall be able to list and filter available donations by food type, distance and expiry time.
FR7	The system shall display donations on an interactive map.
FR8	Receivers shall be able to reserve an available donation; a donation can have only one active reservation.
FR9	Donors shall be able to confirm, complete or cancel reservations; receivers shall be able to cancel their own.
FR10	After a completed reservation, both parties shall be able to submit one rating (1 to 5) and a comment.
FR11	The system shall create in-app notifications for new nearby donations and for reservation status changes.
FR12	The system shall show an impact dashboard calculated from real donation and reservation data.
FR13	Users shall be able to view and edit their profile and view their history.
FR14	Admins shall be able to view, suspend and remove users and listings and review reported content.

2.5 Non-functional requirements

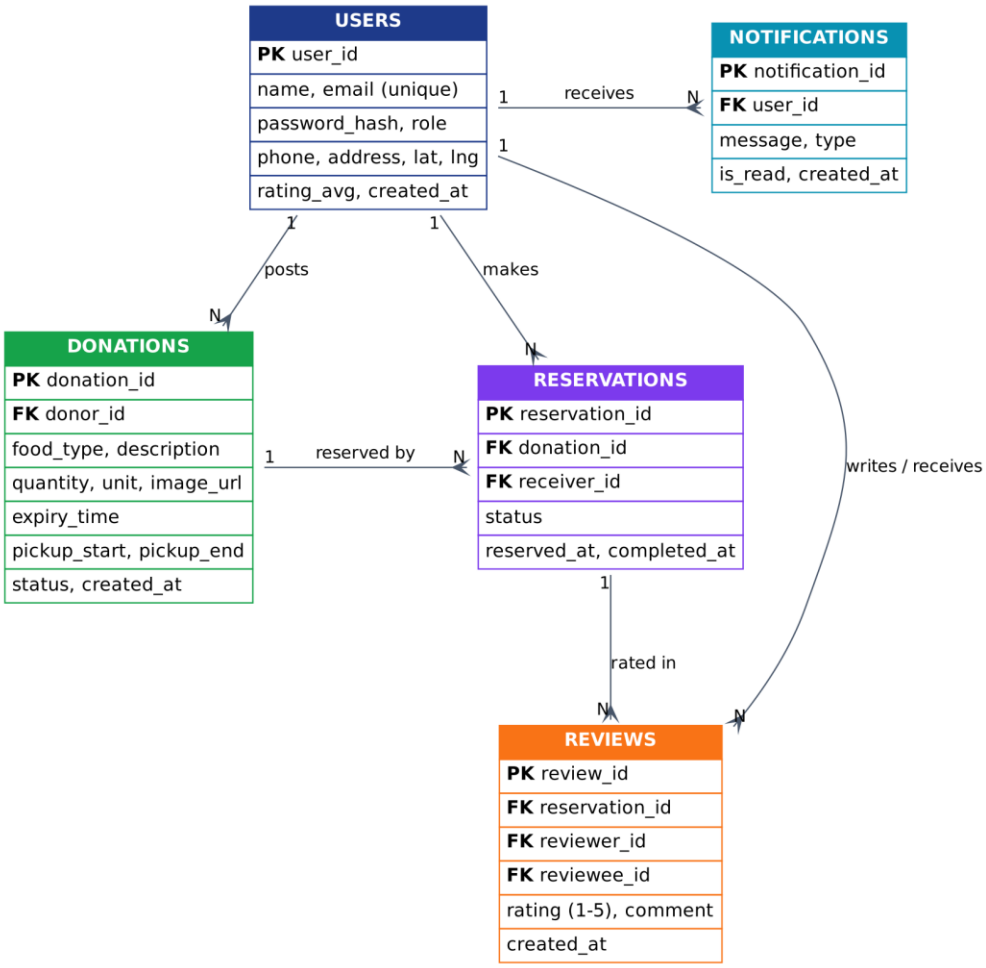
Category	Requirement
Performance	Main pages load in under 3 seconds on a standard connection; 95% of API calls respond in under 500 ms.
Security	Passwords hashed (bcrypt); JWT authentication; role-based authorization; input validation and sanitization; HTTPS in production.
Usability	Clear and simple interface for users with limited technical experience; key actions reachable in three clicks or less.
Accessibility	Text contrast of at least 4.5:1, keyboard navigation, labels on all form fields, alt text on images.
Responsiveness	Layouts adapt to mobile, tablet and desktop widths.
Reliability	Data validated before saving; graceful error messages; database hosted on a managed service with backups.

Category	Requirement
Maintainability	Modular React components, consistent naming conventions, README, and documented API.
Privacy	Users see only the contact details needed to complete a pickup; no personal data is shared publicly.

3. Database Design

3.1 Entity-relationship diagram

The system has five entities. A user can post many donations (as a donor) and make many reservations (as a receiver). Each donation can have many reservation records over time, but only one active at a time. A completed reservation can receive reviews from both parties.



3.2 Relationships

Relationship	Cardinality	Description
USERS – DONATIONS	1 : N	A donor posts many donations
USERS – RESERVATIONS	1 : N	A receiver makes many reservations
DONATIONS – RESERVATIONS	1 : N	A donation can be reserved, cancelled and reserved again; only one active at a time
RESERVATIONS – REVIEWS	1 : N	A completed reservation receives up to two reviews (one from each party)

Relationship	Cardinality	Description
USERS – REVIEWS	1 : N (twice)	A user writes reviews (reviewer) and receives reviews (reviewee)
USERS – NOTIFICATIONS	1 : N	A user receives many notifications

3.3 Logical and physical schema (data dictionary)

USERS

Field	Type	Constraint	Description
user_id	ObjectId	PK	Unique identifier
name	String	Required	Person or organization name
email	String	Unique, required	Login identifier
password_hash	String	Required	Hashed password (never stored in plain text)
role	Enum	donor | receiver | admin	Access level
phone	String	Optional	Contact number for pickup
address	String	Optional	Human-readable address
lat, lng	Number	Optional	Coordinates for map and distance (geospatial index)
rating_avg	Number	Default 0	Average rating received
created_at	Date	Default now	Registration date

DONATIONS

Field	Type	Constraint	Description
donation_id	ObjectId	PK	Unique identifier
donor_id	ObjectId	FK → USERS	Owner of the donation
food_type	String	Required	Category, e.g. cooked meals, bakery, produce
description	String	Optional	Extra details
quantity, unit	Number, Enum	Required	Amount and unit (kg, meal, tray)
image_url	String	Optional	Photo of the food
expiry_time	Date	Required	After this time the donation expires
pickup_start, pickup_end	Date	Required	Pickup time window

Field	Type	Constraint	Description
status	Enum	available | reserved | completed | expired	Current state
created_at	Date	Default now	Publication date

RESERVATIONS

Field	Type	Constraint	Description
reservation_id	ObjectId	PK	Unique identifier
donation_id	ObjectId	FK → DONATIONS	Reserved donation
receiver_id	ObjectId	FK → USERS	Receiver who reserved
status	Enum	pending | confirmed | completed | cancelled	Current state
reserved_at	Date	Default now	Reservation time
completed_at	Date	Optional	Pickup completion time

REVIEWS

Field	Type	Constraint	Description
review_id	ObjectId	PK	Unique identifier
reservation_id	ObjectId	FK → RESERVATIONS	Completed handover being reviewed
reviewer_id	ObjectId	FK → USERS	Author
reviewee_id	ObjectId	FK → USERS	Person being rated
rating	Number	1 to 5	Star rating
comment	String	Optional	Written feedback
created_at	Date	Default now	Submission date

NOTIFICATIONS

Field	Type	Constraint	Description
notification_id	ObjectId	PK	Unique identifier
user_id	ObjectId	FK → USERS	Recipient
message	String	Required	Notification text
type	String	Required	new_donation, reservation_update, review, system

Field	Type	Constraint	Description
is_read	Boolean	Default false	Read state
created_at	Date	Default now	Creation time

3.4 Normalization and design notes

- All tables are in third normal form: each non-key attribute depends only on the key, and repeated data such as donor details is referenced by foreign key instead of copied.
- rating_avg in USERS is a deliberate denormalization to avoid recalculating averages on every profile view; it is updated whenever a review is saved.
- A unique rule on (reservation_id, reviewer_id) prevents duplicate reviews for the same handover.
- Indexes: unique index on users.email; geospatial index on user and donation location; compound index on donations (status, expiry_time) for fast browsing.
- MongoDB is used so coordinates and geospatial queries (nearby donations) are supported natively; the schema could be mapped to PostgreSQL tables one to one.

4. UI/UX Design

4.1 Design principles

- Trustworthy and warm: an earthy palette of deep green and amber on a soft cream background reflects sustainability and community.
- Simple first: both restaurant owners and non-profit staff must complete the main task without training.
- One main action per screen: Donate food, Find food, Reserve now.
- Clear status everywhere: expiry countdowns, reservation badges and notification states are always visible.
- Mobile-first and responsive: every screen works from 360 px up to desktop.

4.2 UI guidelines

Color palette

Swatch	Name	Use
#1B5E3B	Deep green (primary)	Header, primary buttons, key headings
#F59E0B	Amber (accent)	Call-to-action highlights, badges, expiry warnings
#FBF7EE	Cream (background)	Page background
#FFFFFF	White (surface)	Cards and forms
#1F2937	Charcoal (text)	Body text and headings
#DC2626	Red (error)	Errors, cancelled status, destructive actions
#16A34A	Green (success)	Completed status and confirmations

The hex values are the team's working tokens and will be aligned with the final exported designs.

Element	Guideline
Typography	A clean modern sans-serif (for example Inter). Headings semi-bold, body regular; base size 16 px with a clear size scale.
Shape and depth	Rounded cards (12 to 16 px radius), soft shadows, generous white space.
Components	Buttons (primary, secondary, danger), form fields with labels, cards, status badges, tables, modals, toasts, map markers.
Status badges	available (green), reserved (amber), completed (blue), expired / cancelled (grey / red); always text plus color.
Icons and images	Consistent line icons; real food photos on donation cards with a neutral placeholder.
Language and direction	English interface first; layout built so Arabic (right-to-left) can be added later.

4.3 Accessibility

- Text contrast of at least 4.5:1 against its background; color is never the only way to show status.
- All form fields have visible labels and clear error messages; focus states are visible; the app is usable with keyboard only.
- Images have alt text; tap targets are at least 44 px on mobile.
- Plain wording and short sentences for users with limited technical experience.

4.4 Screen inventory

#	Screen	Role	Purpose
1	Landing page	Public	Explain the mission, show impact numbers, buttons for Donate Food and Find Food
2	Login / Register	Public	Sign in or create an account and choose Donor or Receiver
3	Donor dashboard	Donor	Active donations, pending pickups, quick stats, Add New Donation
4	Add donation form	Donor	Food type, quantity, expiry, pickup window, photo, map location picker
5	Browse donations	Receiver	Grid or list of donations, filters sidebar, map toggle
6	Donation detail	Receiver	Full details, donor mini-profile, map, Reserve Now
7	Reservations	Donor, Receiver	Reservations with status badges and actions
8	Receiver dashboard	Receiver	Active reservations, stats, shortcut to browse food, activity feed
9	Impact dashboard	All	Charts for meals saved, environmental impact, top donors
10	User profile	All	Ratings, history, edit profile
11	Notifications	All	List with unread indicators and icons per type
12	Admin panel	Admin	Manage users, donations, reservations and reported content
13	Success / confirmation	All	Confirmation after reserving or completing a pickup
14	404 / error page	All	Friendly error with a link back to the home page

4.5 Main user flows

Flow	Steps
Donor flow	Landing → Register as Donor → Donor dashboard → Add donation → Donation published → Receive reservation notification → Confirm → Mark completed → Review receiver
Receiver flow	Landing → Register as Receiver → Browse donations (list or map) → Donation detail → Reserve Now → Success screen → Pick up food → Review donor
Admin flow	Login → Admin panel → Review users and reported listings → Suspend or remove → Check platform statistics